

FIT2109 Computer Science Workshop

Week 5: Version Control with Git — Collaboration & History

Zahra Mirzamomen

Department of Software Systems and Cybersecurity

Faculty of IT, Monash University

What changes this week

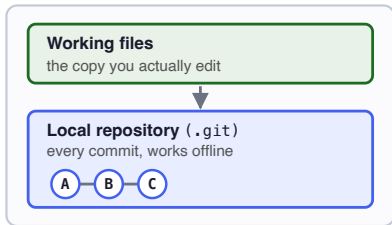
Week 4 — Git on one machine

Commit, branch, merge, stash. One repository, one person, one history you fully controlled.

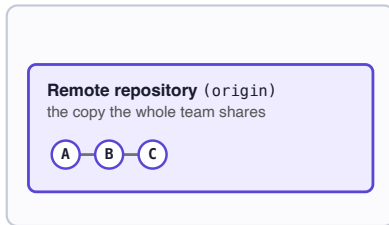
Week 5 — Git between machines

Many clones of one project. Now history is *shared*, and an action of yours can affect somebody else's work.

On your machine



On a server



you sync, explicitly

Nothing crosses the middle on its own — and every teammate has a clone just like the left-hand side.

What you should be able to do by the end

Week 4 taught you commands. This week is about judgement.

- Say what `fetch`, `pull` and `push` each change — before running them.
- Recover from a rejected push, and explain why Git refused it.
- Run the feature-branch and merge-request workflow end to end.
- Resolve a conflict by deciding what the code should say, not by picking a side.
- Choose **merge or rebase** for a situation, and justify it.
- Pick the right lens on history: `log`, `diff`, `blame`, `show`, `bisect`.
- Choose **revert or reset** from one question: has it been shared?

How today works

The same rhythm, four times

A short **recap** of what you read, the **workflow** you would actually type, a **demo** to watch — then you do the **activity** in your group of 3–4.

#	Activity	Time
1	Remotes, Upstream Tracking, and Synchronising Repositories	15 min
2	Branch-Based Collaboration and Merge Requests	15 min
3	Team Merging, Conflict Resolution, and Rebase Decisions	15 min
4	History Analysis and Recovery	15 min

Handout on Ed Lessons. Work on your own machine, discuss as a group.

Clone, Remote, Upstream

1 • A clone is your own complete copy

Cloning gives you the whole project: every file, and its entire history. It sits on your own machine, so you can work on it offline.

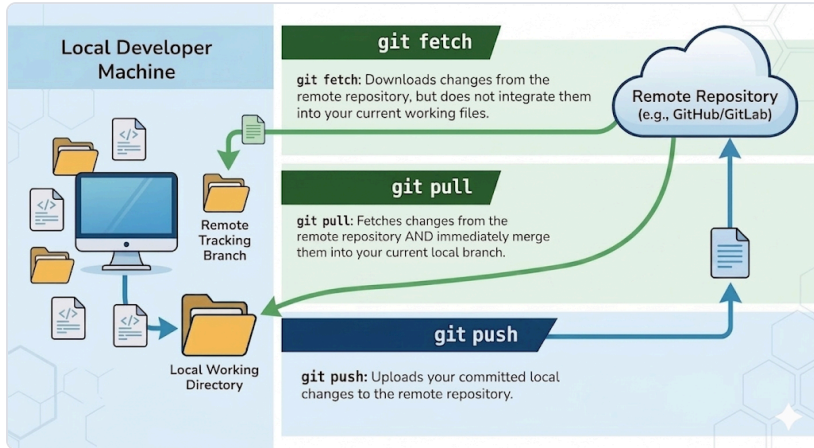
2 • A remote is a copy that lives somewhere else

It is just another repository, usually on a server, that your clone knows the address of. Your clone keeps that address under a short name — by default, `origin`.

3 • An upstream is the branch yours is paired with

Each of your branches can be linked to one branch on the remote. That link is its upstream. Once it is set, Git knows which remote branch yours matches — so it can tell you whether you are ahead of it, behind it, or level.

fetch, pull and push: what moves where



Two directions, three commands — and only one of them touches the files you are editing.

`main` and `origin/main` are two different branches

1 • `main` is the local branch

This is the branch the work happens on.
It moves forward each time a new commit is made locally.

2 • `origin/main` is a note about the remote

It records where `main` stood on the server, the last time the two were in contact.
It is read-only: no commit is ever made on it.

3 • So "up to date" can be out of date

`git status` compares those two — it never contacts the server.
Only `fetch` or `pull` brings the note up to date.

Demo 1: cloning a project from GitLab

Watch first.

Cloning a project from GitLab, then finding our way around a repository none of us has seen before.

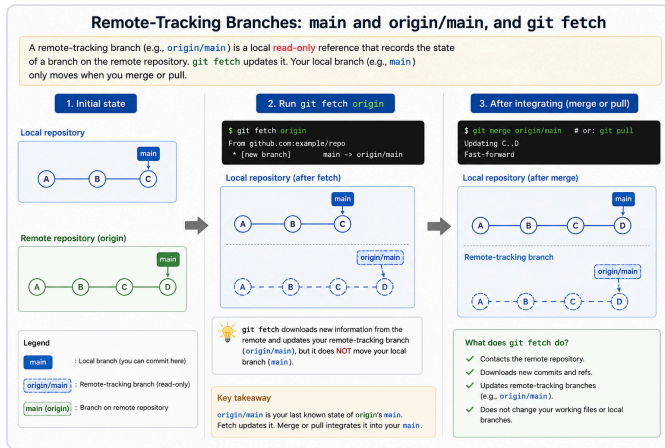
Commands covered

```
git clone <url>
git status
git remote -v
git branch -a
git branch -vv
git log --oneline --graph --all
git show HEAD
```

What to notice

GitLab's **Clone** button offers two URLs: SSH needs a key set up first, HTTPS asks for a token. After cloning, `git log` works straight away with no network — the whole history came along. `git branch -a` also lists the branches that exist on the server.

What `fetch` changes, and what it leaves alone



Only the read-only `origin/main` moves. Your `main` stays put until you merge or pull.

What each command actually changes

Command	Direction	What it does	Local files?	Local branch?	Remote?
<code>git fetch</code>	remote → local	Downloads new commits into remote-tracking branches.	No	No	No
<code>git pull</code>	remote → local	Fetches, then integrates into the current local branch.	Yes	Yes	No
<code>git push</code>	local → remote	Uploads commits already made locally.	No	No	Yes

The practical consequence

`git fetch` is always safe to run — it touches neither the local files, the local branch, nor the remote.
`git pull` is `fetch` plus an integration step, and that step does change the local branch.

Workflow: when a push is rejected

```
$ git push
! [rejected]          main -> main (fetch first)
error: failed to push some refs to 'github.com:group12/week5-lab.git'
```

What it means — even when nothing conflicts

The remote has a commit the local clone has never seen, and accepting this push would strand it. Git is not reporting a mistake — it is refusing to discard someone else's work.

The standard two-step fix

```
git pull  # bring their commit in, resolve anything that clashes
git push  # now your history contains theirs, so it is accepted
```

What not to reach for

`git push --force` makes the message go away by deleting their commit from the remote — the one command here that can lose a teammate's work.

Demo 2: moving commits between two clones

Watch first.

Two clones of one repository, side by side. A commit is made in one and sent to the server; the other collects it — with a look at what arrived before taking it.

Commands covered

```
git remote -v
git branch -vv
git commit
git push
git fetch
git log main..origin/main
git pull
```

What to notice

After `git fetch`, the new commits are already on this machine and can be read — before anything local has moved.

`git pull` is the step that changes the local branch.

Remotes, Upstream Tracking, and Synchronising Repositories

Open the Week 5 handout on **Ed Lessons** and work through Activity 1 in your group of 3–4.

It covers

A clone that has not synchronised since cloning · which pointers move, and when · why a push is refused and how to recover · the upstream-tracking mini scenario.

Workflow: starting work on a shared project

Do this before starting a new feature branch

```
git switch main
git fetch origin                # what happened while I was away?
git log main..origin/main --oneline # read it before taking it
git pull                       # now integrate
git switch -c feature-parser    # branch from a current main
```

Why branch *after* pulling, not before

A branch started from a stale `main` begins life behind, and the gap only grows — starting current is the cheapest conflict prevention there is.

Everyone on the team is doing this at the same time

Each person branches off `main` for the task they are on, so several branches exist side by side. A finished branch is not merged straight into `main` — it is offered for review first. That offer is a **merge request**.

What a merge request is

1 · A request to merge one branch into another

You push a feature branch, then ask for it to be merged into `main`.
Its commits, its diff and the review comments all sit together in that one request.

2 · The two branches it names: base and compare

The **compare** branch holds the new work; the **base** branch is the one it would be merged into.
The diff under review is the difference between those two.

3 · It belongs to the platform, not to Git

There is no `git merge-request` command. GitLab calls it a merge request; GitHub calls the same thing a pull request.
Clone the project elsewhere and the branches come too — the requests do not.

Why teams work this way: `main` stays stable, because nothing reaches it until somebody else has looked at it.

Demo 3: the feature branch and merge request

The full sequence, in order

<code>git switch main && git pull</code>	# 1 · start from a current main
<code>git switch -c feature-readme</code>	# 2 · one branch per task
	# 3 · edit, then git add / git commit
<code>git push -u origin feature-readme</code>	# 4 · publish it
	# 5 · open the merge request in the browser
	# 6 · commit + push again — the request updates
	# 7 · merge on the platform
<code>git switch main && git pull</code>	# 8 · bring the merge back down
<code>git branch -d feature-readme</code>	# 9 · tidy up

Steps 5 to 7 have no Git command at all — they happen in a browser. Everything else is Git.

What makes a merge request easy to review

Do this	Because
One task per branch	A reviewer can hold one idea in their head. A branch doing three things gets skimmed, not reviewed.
Keep it small	Review quality falls off a cliff past a few hundred lines. Two small requests beat one large one.
Say why in the description	The diff shows <i>what</i> changed. Only you can supply <i>why</i> — and that is what review is for.
Keep the branch current	Update from <code>main</code> as you go, so the merge is not a surprise on the last day.
Delete it after merging	The commits live on in <code>main</code> ; the branch name is just clutter once merged.

Naming helps too: `feature-parser`, `fix-login-timeout` — a name that says what, not `test2`.

Branch-Based Collaboration and Merge Requests

Back to the handout — Activity 2, same groups.

It covers

Two students on one GitLab project: an eight-step workflow sketch, and what exists *where* at each step · base versus compare · why the request updates itself · where a merge request actually lives.

A conflict is where Git cannot decide

A conflict is the exception, not the rule

The **same region of the same file** changed differently on both sides.
Any automatic choice would throw away somebody's work, so Git stops and asks.

```
<<<<<<< HEAD
TIMEOUT = 30      # Ana raised it - the API was timing out
=====
TIMEOUT = 5       # Ben lowered it - the tests were slow to fail
>>>>>>> feature-fast-tests
```

Top — HEAD

The version already on the branch being worked on.

Bottom — incoming

The version being merged in, labelled with its branch.

The answer does not have to be either side

Both changes are correct — Git is asking what the code should now say, not which side wins.

Workflow: resolving a conflict

The loop, start to finish

```
git status          # 1 · which files are unmerged? (can be several)
                    # 2 · edit each one; delete all three marker lines
git add app.py      # 3 · this IS how you mark it resolved
git status          # 4 · repeat 2-3 until nothing is unmerged
git merge --continue # 5 · records the merge commit
```

Resolving right now is not compulsory

```
git merge --abort
```

What a rebase does

The command

`git rebase <base>` takes the commits on the branch you are on and applies them again, one at a time, on top of `<base>`.

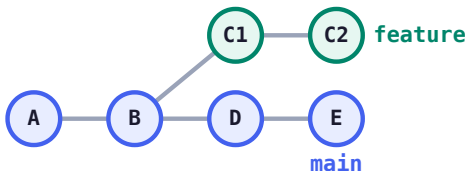
Which branch moves

```
git switch feature  
git rebase main
```

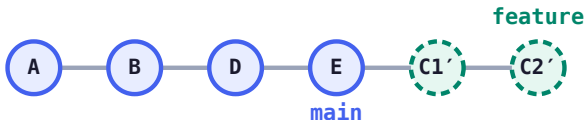
What comes out

One straight line, and no merge commit.
`main` itself never moves.

Before

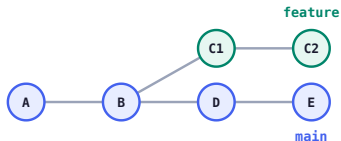


After `git rebase main`

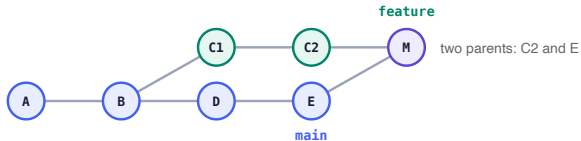


Merge vs rebase, illustrated

1 · Starting point — feature branched off at B, then main moved on



2 · `git merge main` — the fork is preserved



3 · `git rebase main` — one straight line, new commit identities



Why rewriting shared history is risky

1 · A rebase makes new commits

The changes and the messages stay the same, but every commit gets a new ID.
The originals are not edited — they are replaced by copies, and the old ones are left behind.

2 · That is a problem once others have those commits

Suppose a teammate started their own branch from `feature-report` yesterday, and it is then rebased and force-pushed.
Their commits now point back to commits that no longer exist on the server, so their next pull brings duplicates and conflicts nobody caused.

3 · The golden rule

Never rebase commits that others may already have.
A tidy history is not worth breaking someone else's work.

Workflow: keeping a feature branch current

Your own branch, not yet shared — rebase

```
git fetch origin
git switch feature-ui    # stand on the branch that will move
git rebase origin/main  # replay my work on top of current main
```

A branch you share with someone — merge

```
git fetch origin
git switch feature-ui    # same starting point
git merge origin/main    # nobody's commits get rewritten
```

Demo 4: integrating with rebase instead of merge

Watch first.

A feature branch that has fallen behind `main`, brought up to date by rebasing.

Commands covered

```
git rebase main
git rebase --continue
git rebase --abort
git log --oneline --graph
      --decorate --all
```

What to notice

A rebase pauses *once per conflicting commit*, so it can stop more than once — that is the loop, not a failure. And it ends with `--continue`, **never** `git commit`.

Merge vs rebase, side by side

	<code>git merge</code>	<code>git rebase</code>
Resulting files	The same, given the same conflict decisions — never what you are choosing between.	
History shape	Fork preserved; adds a merge commit.	One straight line; no merge commit.
Commit identity	Existing commits untouched.	Commits are replaced by new copies.
Conflicts	At most one pause.	Can pause once per replayed commit.
Records	The true story: two lines met here.	A tidy story: as if you never diverged.
Safe when	Always, shared branches included.	The branch is yours alone .

Rule of thumb: rebase before sharing, merge after.

Team Merging, Conflict Resolution, and Rebase Decisions

Handout, Activity 3 — three scenarios and a decision table.

It covers

- A** — a conflict in `app.py`: markers, resolution, and whether the answer is even one of the two sides.
- B** — rebasing a private feature branch. **C** — the risky rewrite of a shared branch, and the merge-or-rebase decision table.

Four ways to look at history

Each answers a different shape of question

- `git log` — *which commits?*
- `git diff` — *what text changed?*
- `git blame` — *who last touched this line, and when?*
- `git show` — *what did one commit do, and why?*

The skill is choosing, not memorising

Real investigations chain two or three of them together. Almost nobody uses these one at a time.

git bisect: binary search over commits

The problem it solves

It worked last week, it fails now, and the code changed many times in between. A **regression** is behaviour that used to work and no longer does.

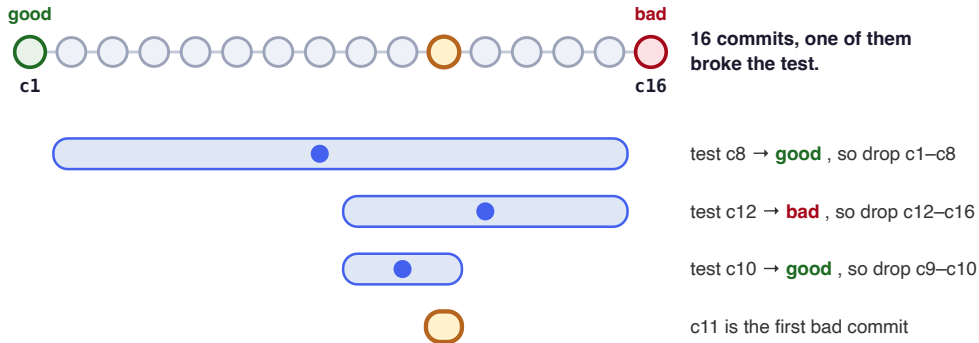
How it works

You name one *good* commit and one *bad* one. Git checks out the midpoint *for you to test* — real code you can actually run — and each verdict discards half the remaining range.

Different question from `blame`

Blame asks "which commit last changed this *line*?" Bisect asks "which commit changed the *behaviour* I am observing?" — which may involve no single line at all.

Why four answers are enough for sixteen commits



4 tests for 16 commits · 10 for 1,000 · 20 for a million — $\log_2(n)$, the same halving you met in searching and sorting.

Workflow: a bisect session

Start, answer, finish

```
git bisect start
git bisect bad           # the current commit is broken
git bisect good v1.0     # a known-good commit – here, the v1.0 release tag
                        # Git checks out a midpoint – run your test, then:
git bisect good          # ...or bad. Repeat until Git names the culprit
git bisect reset         # ALWAYS – puts you back on your branch
```

If you forget `reset`

You are left on a detached midpoint commit, not your branch — and commits made there are easy to lose.

If the test can be scripted

```
git bisect run pytest -x
# Git answers for you
```

When bisect can be trusted

It assumes the bug appears once and then stays

A bug that only shows up **some of the time** breaks that assumption.

Answer "good" once on a commit that was really bad, and bisect will name the wrong commit — with no sign that anything went wrong.

So test the same way every time

Use one command, ideally a scripted test, and never change what you are checking partway through the session.

Small commits make the answer useful

Bisect points at exactly one commit.

If that commit did one thing, that is the answer. If it bundled a fix, a refactor and a reformat together, it has only narrowed the search to 400 lines.

Demo 5: a live bisect, start to reset

Watch first.

A repository of seventeen commits with a test that passes at the start and fails at the tip. Nobody knows which commit broke it.

Commands covered

```
git bisect start  
git bisect bad / good  
git show  
git bisect reset
```

What to notice

Git *checks out* each commit, so the test runs against real code. Count the answers — seventeen commits should take about four.

Undoing a commit: one question decides how

Has the commit been pushed yet?

This is not a matter of preference.

The answer decides which of the two commands is safe to use.

Shared → `git revert`

Adds a *new* commit that cancels the old one.
History only grows, so your branch stays ahead of the remote and a plain `push` works.
A teammate who already pulled the bad commit simply pulls one more.

Still local → `git reset`

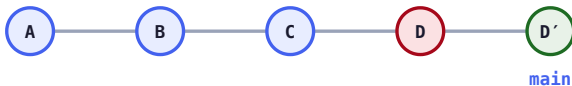
Moves the branch pointer itself, taking the commit off the branch entirely.
Tidier, and perfectly safe while nobody else has the commit.
Reset one you *have* already pushed and you end up behind the remote, with the next push refused.

revert adds; reset removes

Starting point — D is the bad commit



git revert D — history grows; nothing is removed



D' undoes what D did.
Still ahead of the remote,
so a normal push works.

git reset --hard HEAD~1 — the branch moves back



D is abandoned. Fine while D
was never pushed — if it was,
you are now behind the remote.

History Analysis and Recovery

Handout, Activity 4 — the last one, then the reflection.

It covers

A — eight commands and eight questions to match up. **B** — a regression traced with `bisect`: how many steps, and what would fool it. **C** — four recovery situations, decided by one question.

Four decisions to take away

The question	How to answer it
Take work in, or just look at it?	<code>fetch</code> to look — nothing of yours moves. <code>pull</code> when you are ready to integrate.
Push to <code>main</code>, or open a request?	The branch carries the work, the merge request carries the conversation. One is in the repository, the other only on the platform.
Merge, or rebase?	Same files either way. Rebase while the branch is yours alone; merge once it is shared.
Revert, or reset?	Shared → <code>revert</code> , so history only grows. Still local → <code>reset</code> , which takes the commit off the branch entirely.

Notice how often the same thing settles it: whether anyone else already has your commits.

Exit ticket

Before next time

Practise on a repository that is genuinely shared — a group assignment, or a repo with a friend. Everything today only bites when a second person is involved.

Check yourself

Section 5.8 of the pre-reading is ten questions covering the whole week. If any answer is a guess, that is the section to reread.

Exit ticket

Wrap up with the PollEv activity!